

CounterStock: Developer Guide

Winter Semester 2026

Architecture, Concurrency & Extension Guide

THGA Bochum

Instructor: Stephan Bökelmann

Informationstechnik und Digitalisierung

Table of Contents

1	System Architecture	2
2	Database Schema and Relational Constraints	2
3	Concurrency Control & Race Condition Prevention	2
4	Step-by-Step Guide: Adding a New Feature	2
4.1	Step 1: Backend Endpoint Implementation	3
4.2	Step 2: API Client Expansion	3
4.3	Step 3: Frontend Interface Action	3
4.4	Step 4: Acceptance Test Validation	3
5	System Upgrade Roadmap	3
6	Frequently Asked Questions (FAQ)	4

Developer Note: This technical specification covers relational integrity rules, transaction bounds with `SELECT ...FOR UPDATE`, the testing suite, and a 4-step walkthrough for extending the API and UI.

1 System Architecture

CounterStock implements an enterprise 3-tier structure designed for relational data integrity, headless service consumption, and multi-client scale.

3-Tier Architecture Overview:

- **Data Tier:** PostgreSQL 16 (Third Normal Form) running inside an isolated Docker network.
- **Application Tier:** Asynchronous FastAPI gateway managing connection pooling via `psycopg2.extras.RealDictCursor`.
- **Presentation Tier:** Decoupled Python Tkinter client packaged into a standalone Debian installer.

2 Database Schema and Relational Constraints

The relational model normalizes stock, menus, recipes, and transactions across 6 entities.

```
1  -- Integrity constraints from init.sql
2  CREATE TABLE ingredients (
3      id SERIAL PRIMARY KEY,
4      name VARCHAR(100) NOT NULL UNIQUE,
5      stock_quantity NUMERIC(10, 2) NOT NULL CHECK (stock_quantity >= 0),
6      unit VARCHAR(20) NOT NULL,
7      reorder_threshold NUMERIC(10, 2) NOT NULL DEFAULT 5.0
8  );
9
10 CREATE TABLE recipes (
11     menu_item_id INT REFERENCES menu_items(id) ON DELETE CASCADE,
12     ingredient_id INT REFERENCES ingredients(id) ON DELETE RESTRICT,
13     quantity_required NUMERIC(10, 2) NOT NULL CHECK (quantity_required > 0),
14     PRIMARY KEY (menu_item_id, ingredient_id)
15 );
```

3 Concurrency Control & Race Condition Prevention

A central risk in POS gastronomy is concurrent requests competing for limited stock. CounterStock resolves race conditions using pessimistic row-level locking:

```
1  -- Atomic row-locking during order deduction
2  SELECT id, name, stock_quantity
3  FROM ingredients
4  WHERE id = ANY(%s)
5  FOR UPDATE;
```

Locking Mechanism: The `FOR UPDATE` clause locks the selected ingredient rows in the PostgreSQL engine. Concurrent transactions attempting to read or decrement the same rows wait until the active transaction commits or rolls back, completely eliminating phantom stock reads and overselling.

4 Step-by-Step Guide: Adding a New Feature

To illustrate how the system can be extended, this guide walks through adding an **Ingredient Deletion Feature**.

4.1 Step 1: Backend Endpoint Implementation

Open backend/main.py and define the authenticated endpoint:

```

1 @app.delete("/ingredients/{ing_id}", dependencies=[Depends(verify_token)])
2 def delete_ingredient(ing_id: int, conn=Depends(get_db)):
3     with conn:
4         with conn.cursor() as cur:
5             cur.execute(
6                 "DELETE FROM ingredients WHERE id = %s RETURNING id;",
7                 (ing_id,)
8             )
9             deleted = cur.fetchone()
10            if not deleted:
11                raise HTTPException(status_code=404, detail="Ingredient not found")
12            return {"status": "success", "deleted_id": ing_id}

```

4.2 Step 2: API Client Expansion

In frontend/src/db_frontend/api.py, expose the call:

```

1 def delete_ingredient(ing_id: int):
2     r = requests.delete(f"{BASE_URL}/ingredients/{ing_id}", headers=HEADERS,
3                         timeout=5)
4     if r.status_code != 200:
5         raise ValueError(f"HTTP Error {r.status_code}: {r.text}")
6     return r.json()

```

4.3 Step 3: Frontend Interface Action

In frontend/src/db_frontend/ui.py, bind a button with an explicit confirmation dialog:

```

1 def _on_delete_click(self):
2     if messagebox.askyesno("Confirm", "Are you sure you want to delete this
3         ingredient?"):
4         api.delete_ingredient(self.selected_ing_id)
5         self._refresh()

```

4.4 Step 4: Acceptance Test Validation

In backend/test_api.py, write the test case:

```

1 def test_delete_ingredient_auth_protection():
2     res = requests.delete(f"{BASE_URL}/ingredients/1")
3     assert res.status_code == 401

```

5 System Upgrade Roadmap

The decoupled design supports several clean upgrade paths:

- **Kitchen Display WebSockets:** Migrate order event propagation from polling to push-based WebSockets.
- **Role-Based Access Control (RBAC):** Upgrade basic static tokens to signed JSON Web Tokens (JWT) carrying cashier versus supervisor scopes.
- **Automated Purchase Orders:** Celery worker task that queries low-stock items nightly and generates automated supplier order drafts.

6 Frequently Asked Questions (FAQ)

Q: Why use psycopg2 instead of an ORM like SQLAlchemy?

Raw SQL gives full, predictable control over transaction boundaries, isolation levels, and `SELECT ... FOR UPDATE` row locks without unintended abstraction layers.

Q: Why use NUMERIC(10,2) instead of FLOAT for inventory?

Floating-point numbers introduce binary rounding inaccuracies. `NUMERIC(10,2)` guarantees exact fractional arithmetic for weight and volume.

Q: How is the desktop client packaged?

PyInstaller freezes the application and its dependencies into a standalone directory, which `fpm` bundles into a standard Debian archive.